

PROMPT COMPLET POUR KIRO AI — MODULE TELEGRAM FOURNISSEURS

Document de spécification et prompt d'implémentation pour Kiro AI

1. OBJECTIF PRINCIPAL

Créer dans le Backoffice un module permettant de :

- connecter et gérer une session Telegram utilisateur via MTPROTO ;
- démarrer et arrêter le collecteur Telegram depuis l'interface du Backoffice ;
- choisir les canaux fournisseurs à surveiller ;
- détecter automatiquement les nouveaux messages ;
- récupérer les textes, photos, albums, vidéos et documents pertinents ;
- associer chaque message à son fournisseur et à son canal source ;
- afficher immédiatement les produits/articles reçus dans le Backoffice ;
- stocker durablement les données et médias dans Supabase ;
- préparer ces données pour les futurs workflows n8n et la publication sur les réseaux sociaux.

Le système doit être conçu comme un véritable module de production et non comme un simple script de test.

2. ARCHITECTURE ATTENDUE

Architecture cible :



Le collecteur Telegram ne doit pas dépendre de l'ouverture permanente de l'interface web.

Créer un service backend dédié capable de fonctionner en continu. Pour le développement local, Termux peut servir à tester le collecteur, mais l'architecture finale doit permettre son déploiement sur un serveur.

3. TECHNOLOGIE TELEGRAM

Utiliser MTPROTO avec Telethon (Python).

Le système doit utiliser une session utilisateur Telegram, car nous devons pouvoir lire les messages des canaux auxquels le compte est membre.

NE JAMAIS exposer publiquement :

- api_hash ;
- session Telegram ;
- code de connexion ;
- tokens ;
- clés Supabase ;
- secrets.

Tous les secrets doivent être stockés dans des variables d'environnement ou un système sécurisé.

Prévoir une configuration avec :

TELEGRAM_API_ID
TELEGRAM_API_HASH

La session Telegram doit être gérée de manière sécurisée côté serveur.

4. MODULE BACKOFFICE À CRÉER

Ajouter dans le menu du Backoffice une section :

■ Telegram Suppliers Hub

Cette section doit contenir les pages suivantes :

A. Dashboard Telegram

- statut du collecteur : En ligne / Arrêté / Erreur ;
- bouton Démarrer ;
- bouton Arrêter ;
- dernier message reçu ;
- nombre de canaux surveillés ;
- nombre de fournisseurs ;
- nombre de produits/messages reçus ;
- erreurs récentes ;
- activité récente.

B. Canaux fournisseurs

Afficher tous les canaux accessibles par le compte Telegram.

Pour chaque canal :

- nom ;
- identifiant Telegram ;
- photo/avatar si disponible ;
- statut surveillé ou non ;
- fournisseur associé ;
- nombre de produits/messages collectés ;
- dernier message ;
- bouton Activer/Désactiver.

Exemple de canal déjà utilisé pour les tests :

■■■■■■■ ecom-Taj ■■■■■■■■0601628220

ID : -1001155338501

Mais le système doit être générique pour ajouter d'autres canaux plus tard.

5. GESTION DES FOURNISSEURS

Créer ou utiliser une table Suppliers/Fournisseurs.

Un fournisseur peut être associé à un ou plusieurs canaux Telegram.

Informations fournisseur :

- id ;
- nom ;
- téléphone ;
- WhatsApp ;
- canal Telegram ;
- Telegram channel id ;
- adresse éventuelle ;
- notes ;
- statut actif/inactif ;
- date de création.

L'utilisateur doit pouvoir :

- créer un fournisseur ;
- modifier un fournisseur ;
- associer un canal Telegram ;
- changer l'association ;
- désactiver un fournisseur.

6. COLLECTE DES MESSAGES

Le collecteur doit écouter les nouveaux messages des canaux activés.

Pour chaque nouveau message, enregistrer :

Informations Telegram :

- message_id ;
- channel_id ;
- channel_name ;
- sender_id si disponible ;
- date ;
- texte original ;
- lien/permalink si disponible ;
- type de message ;
- grouped_id pour identifier les albums.

Types à gérer :

- message texte ;
- photo ;
- plusieurs photos/albums ;
- vidéo ;
- document ;
- message avec texte + média.

Le système doit éviter les doublons grâce à une contrainte unique basée au minimum sur :
(channel_id + message_id).

Le collecteur doit pouvoir redémarrer sans perdre son historique ni recréer les mêmes enregistrements.

7. GESTION DES IMAGES ET MÉDIAS

Les médias Telegram doivent être récupérés automatiquement.

Fonctionnement attendu :

1. Détection du message.
2. Identification du média.
3. Téléchargement temporaire si nécessaire.
4. Upload vers Supabase Storage.
5. Enregistrement de l'URL ou du chemin dans la base de données.
6. Suppression du fichier temporaire après succès.

Pour chaque média enregistrer :

- id ;
- product_message_id ;
- type : image/video/document ;
- storage_path ;
- public_url ou signed URL selon la stratégie ;
- nom original ;
- taille ;
- ordre d'affichage ;
- metadata.

Les albums Telegram doivent être regroupés comme une seule publication/fiche produit lorsque les messages appartiennent au même grouped_id.

8. INTERFACE DES PRODUITS / ARTICLES REÇUS

Créer une page :

■ Produits reçus depuis Telegram

Chaque publication fournisseur doit apparaître sous forme de carte ou de ligne avec :

- image principale ;
- galerie d'images ;
- nom provisoire ou titre ;
- texte original Telegram ;
- fournisseur ;
- canal source ;
- date ;
- statut.

Statuts :

- Nouveau ;
- À analyser ;
- En attente de validation ;
- Validé ;
- Prêt à publier ;
- Publié ;
- Ignoré.

Actions :

- voir le détail ;
- modifier les informations ;
- extraire ou saisir le prix ;
- choisir les images ;
- supprimer une image ;

- ajouter des informations ;
- associer à une catégorie ;
- valider ;
- envoyer vers le workflow de publication ;
- ignorer.

Conserver toujours le message original Telegram pour la traçabilité.

9. EXTRACTION INTELLIGENTE DES INFORMATIONS

Prévoir une couche d'analyse permettant, progressivement, d'extraire depuis le texte Telegram :

- nom du produit ;
- prix ;
- prix de gros ;
- quantité minimale ;
- couleur ;
- taille ;
- caractéristiques ;
- téléphone ;
- WhatsApp ;
- disponibilité ;
- ville/localisation.

Cette extraction doit être robuste face aux messages en :

- arabe ;
- français ;
- anglais ;
- mélange arabe/français.

Au début, l'analyse peut être basée sur des règles simples. Concevoir l'architecture pour permettre plus tard l'ajout d'un agent IA.

10. BASE DE DONNÉES SUPABASE

Créer ou adapter les tables nécessaires.

Tables recommandées :

telegram_connections

- id
- user/account reference
- status
- phone masked
- created_at
- updated_at

telegram_channels

- id
- telegram_channel_id
- name
- username nullable
- supplier_id nullable
- is_monitored
- last_message_at
- created_at

suppliers

- id
- name
- phone
- whatsapp
- notes
- is_active
- created_at

telegram_messages

- id
- telegram_message_id
- telegram_channel_id
- supplier_id
- grouped_id nullable
- text
- message_date
- message_type
- raw_metadata JSONB
- created_at

telegram_media

- id
- telegram_message_id reference
- media_type
- storage_path
- file_name
- mime_type
- file_size
- display_order
- created_at

supplier_products

- id
- supplier_id
- source_message_id
- title
- description
- price nullable
- category_id nullable
- status
- extracted_data JSONB
- created_at
- updated_at

Adapter ces tables au schéma existant de DigitalEcomLand plutôt que créer inutilement des doublons.

11. API BACKEND

Créer des endpoints sécurisés pour :

GET /api/telegram/status

POST /api/telegram/start

POST /api/telegram/stop

GET /api/telegram/channels
POST /api/telegram/channels/:id/monitor
POST /api/telegram/channels/:id/unmonitor

GET /api/telegram/messages
GET /api/telegram/messages/:id

GET /api/telegram/products
GET /api/telegram/products/:id
PATCH /api/telegram/products/:id

GET /api/suppliers
POST /api/suppliers
PATCH /api/suppliers/:id

Les endpoints doivent respecter le système d'authentification et les rôles administratifs déjà présents dans l'application.

12. INTÉGRATION N8N

Le module Telegram ne doit pas dépendre directement de n8n pour fonctionner.

Cependant, préparer une intégration future :

Quand un produit est validé :

Telegram Product

↓

Webhook / Event

↓

n8n

↓

Traitement automatique

↓

Création contenu réseaux sociaux

↓

Publication Facebook Pages / autres plateformes.

Prévoir un mécanisme d'événements ou webhooks internes propre et fiable.

13. GESTION DU COLLECTEUR

Le Backoffice doit pouvoir afficher l'état réel du service.

États possibles :

- STOPPED
- STARTING
- RUNNING
- ERROR
- RECONNECTING

Le collecteur doit :

- se reconnecter automatiquement en cas de problème réseau ;
- journaliser les erreurs ;
- ne pas planter définitivement après une erreur ;
- éviter les doublons ;
- reprendre correctement après redémarrage.

IMPORTANT :

Le bouton « Démarrer » ne doit pas simplement lancer un processus temporaire attaché à une requête HTTP. Utiliser une architecture de worker/service adaptée au déploiement.

14. SÉCURITÉ

Exigences obligatoires :

- aucune clé dans le code source ;
- aucun api_hash dans le frontend ;
- session Telegram inaccessible publiquement ;
- Supabase Service Role uniquement côté serveur ;
- contrôle d'accès administrateur ;
- validation des entrées ;
- logs sans secrets ;
- protection contre les doublons ;
- limitation des fichiers et validation MIME si nécessaire.

Ne jamais committer les fichiers suivants :

*.session
.env
.env.local
credentials.json

15. UX / DESIGN

Le module doit être cohérent avec le design actuel de DigitalEcomLand.

Créer une interface professionnelle et simple :

Menu Telegram

- ■ ■ Dashboard
- ■ ■ Canaux
- ■ ■ Fournisseurs
- ■ ■ Produits reçus
- ■ ■ Activité / Logs
- ■ ■ Paramètres

Sur la page Produits reçus :

- recherche ;
- filtres fournisseur ;
- filtres canal ;
- filtres statut ;
- filtre date ;
- pagination ;
- vue cartes et/ou tableau.

Les nouvelles publications doivent apparaître rapidement dans l'interface, idéalement via Supabase Realtime ou un mécanisme de rafraîchissement adapté.

16. PLAN D'EXÉCUTION DEMANDÉ À KIRO AI

Avant de modifier le code :

1. Analyser minutieusement l'architecture actuelle du dépôt solver492/Digitalecomland.

2. Identifier :

- frontend ;
- backend ;
- système d'authentification ;
- schéma Supabase ;
- tables produits existantes ;
- stockage médias existant ;
- conventions de code ;
- système de rôles.

3. Réutiliser l'existant autant que possible.

4. Ne pas casser les fonctionnalités actuelles.

5. Proposer les migrations nécessaires.

6. Implémenter le module progressivement.

7. Tester chaque étape.

Ordre recommandé :

PHASE 1 — Analyse du projet existant

PHASE 2 — Schéma base de données / migrations

PHASE 3 — Service Telegram MTPROTO

PHASE 4 — API backend

PHASE 5 — Interface Backoffice

PHASE 6 — Téléchargement médias + Supabase Storage

PHASE 7 — Gestion fournisseurs

PHASE 8 — Produits et validation

PHASE 9 — Logs, monitoring et robustesse

PHASE 10 — Préparation intégration n8n

17. CRITÈRES DE RÉUSSITE

Le travail sera considéré comme réussi lorsque :

- ✓ Je peux ouvrir « Telegram Suppliers Hub » dans mon Backoffice.
- ✓ Je peux voir les canaux accessibles.
- ✓ Je peux sélectionner les canaux à surveiller.
- ✓ Le collecteur peut fonctionner de manière fiable.
- ✓ Un nouveau message fournisseur est détecté.
- ✓ Son texte est enregistré.
- ✓ Ses images sont récupérées.
- ✓ Les images sont envoyées vers Supabase Storage.
- ✓ Le fournisseur est identifiable.
- ✓ Le produit/message apparaît dans le Backoffice.
- ✓ Je peux consulter toutes les informations.
- ✓ Je peux modifier et valider le produit.
- ✓ Le système évite les doublons.
- ✓ Le système peut accueillir plusieurs fournisseurs et plusieurs canaux.
- ✓ L'architecture est prête pour n8n et les autopublications.

INSTRUCTION FINALE À KIRO AI

Commence maintenant par analyser complètement le dépôt actuel DigitalEcomLand avant toute modification.

Ensuite, produis un plan précis des fichiers qui doivent être créés ou modifiés, puis implémente le système en respectant l'architecture existante.

Ne remplace pas brutalement le système actuel. Intègre le nouveau module proprement.

Si certaines technologies du dépôt rendent Telethon/Python difficile à intégrer directement, créer un microservice Telegram séparé, connecté de manière sécurisée au backend principal.

Priorité absolue :

FIABILITÉ → SÉCURITÉ → INTÉGRATION AVEC LE BACKOFFICE → SUPABASE → ÉVOLUTIVITÉ.

Le résultat final doit transformer DigitalEcomLand en une plateforme capable de centraliser automatiquement les produits publiés par les fournisseurs dans Telegram.